

Python Assignment 01

Python Logic & Data Structures

Total Marks: 30

Instructions:

- Attempt all questions in the given order.
- Write clean, readable and well-commented Python code with proper explanations of each step.
- Submit your assignment in .ipynb format or as a Google Colab file.

Section A: (5 Marks)

1. Write a program that prints your name, age, and favorite hobby using `print()`.
2. Create two variables and print their sum, difference, product, and division.
3. Create variables of different data types (int, float, string, boolean) and print their types using `type()`.
4. Create a variable containing a number and use `if-else` to check whether it is positive or negative.
5. Create a list of 5 fruits and print:
 - First fruit
 - Last fruit
 - Entire list

Section B: (5 Marks)

1. Create a variable `age` and print:
 - "Child" if age is less than 13
 - "Teen" if age between 13–19
 - "Adult" if age is greater or equal to 20
2. Create a dictionary of a student with keys: name, age, course, marks.
 - Print each value separately
 - Update marks value
3. Create a string variable and:
 - Prints its length
 - Prints first 3 characters and last 3 characters (slicing)
4. Create two sets and perform:
 - Union
 - Intersection
 - Difference

5. Create a variable `number` and check:
- If it is divisible by both 3 and 5
 - Or only by 3
 - Or only by 5

Section C: (15 Marks)

1. **Book Dictionary** (5 marks)

Create a dictionary named `book` with keys `'title'`, `'author'`, and `'price'`. Print the book's title and author.

Example:

Input → `{'title': 'Python Basics', 'author': 'John Smith', 'price': 1500}`

Output → `Title: Python Basics, Author: John Smith`

2. **Fruits Store** (5 marks)

Create a tuple containing five fruit names. Access and print the first and last elements. Then try to change one of the fruits and observe the output.

3. **Grading System** (5 marks)

Create a variable `'marks'` and print grade according to:

- 80 and above → "A"
- 60 to 79 → "B"
- 40 to 59 → "C"
- Below 40 → "Fail"

Section D: Leetcode Questions

1. Two Sum (1 point)
Given an array of integers, return indices of the two numbers such that they add up to a specific target.

 Hint:

Use a loop to check each number

For each number, check if `(target - current_number)` exists in the array

Think about using a dictionary to store values for faster lookup

2. Palindrome Number *(1 point)
Check whether an integer is a palindrome or not.

3. Fizz Buzz *(1 point)
Print numbers from 1 to n:
 - Multiples of 3 → "Fizz"
 - Multiples of 5 → "Buzz"
 - Both → "FizzBuzz"

 Hint:

Loop from 1 to n

Use conditions:

- a. divisible by 3 → "Fizz"
- b. divisible by 5 → "Buzz"
- c. both → "FizzBuzz"

Order of conditions matters

4. Valid Anagram *(1 point)
Check whether two strings are anagrams of each other.

5. Remove Duplicates from Sorted Array *(1 point)
Remove duplicates in-place and return the new length of the array.

Professional Guide to Improving Coding Skills

- **Focus on Concepts First:** Understand programming fundamentals thoroughly before moving to exercises or projects.
- **Gradual Implementation:** Apply learned concepts step by step through small problems, exercises or projects.
- **Observe & Create Report:** After solving each problem observe concepts & approach and create report treat as insights to deepen understanding.
- **Practice Consistently:** Solve coding challenges regularly to strengthen skills and build confidence.
- **Analyze and Learn from Others:** Read and study code from open-source projects or experienced programmers to explore different problem-solving methods.
- **Debugging and Problem-Solving:** Learn to read errors carefully, break problems into smaller parts and improve logical thinking.

- **Work on Projects:** Gradually move from small applications to more complex projects to apply concepts in real scenarios.
- **Participate in Challenges and Communities:** Engage in coding competitions and online communities to expose yourself to diverse problems and collaborative learning.
- **Use Insights for Continuous Improvement:** Treat every observation, error and reflection as an opportunity to strengthen conceptual clarity and problem-solving ability.
- **Consistency & Discipline:** Regular focused practice and reflective learning are key to steady progress and achieving coding goals.

